AEGISAI ENTERPRISE EDITION

SUITE DE DOCUMENTACIÓN TÉCNICA

Guía de Desarrollo y Mantenimiento

Estándares de Código, Pruebas de Seguridad y Extensibilidad

Versión de Plataforma:	**2.5.0 Enterprise Edition**
Alcance Normativo:	**OWASP AISVS v1.0, ISO/IEC 42001:2023, Google Cloud Security**
Licencia:	**Apache License 2.0 (Open Source)**

AEGIS AI ASSURANCE PLATFORM

Guía de Desarrollo y Mantenimiento del Código

Versión: 2.5.0 Enterprise (Edición 2026)

Clasificación: Guía Técnica para Desarrolladores de Software e Ingenieros de IA

Stack Tecnológico: TypeScript 5.8, React 19, Node.js 22, Express, Vite 6, Tailwind CSS v4, Google GenAI SDK

---

1. Configuración del Entorno de Desarrollo Local

1.1 Requisitos de Desarrollo

- **Node.js:** Versión >= 20.10.0 (Recomendado: Node 22.x LTS).
- **Gestor de Paquetes:** npm >= 10.x.
- **IDE Recomendado:** VS Code o Google Antigravity IDE con extensiones de TypeScript, Tailwind CSS y ESLint.

1.2 Puesta en Marcha en Modo Desarrollo (HMR)

```
# 1. Instalar dependencias completas
npm install

# 2. Configurar el archivo de entorno local
cp .env.example .env
# Configurar GEMINI_API_KEY con una clave válida de desarrollo

# 3. Iniciar el servidor en modo desarrollo
npm run dev
```

El comando `npm run dev` ejecuta `tsx server.ts`, el cual activa Vite en modo middleware con **Hot Module Replacement (HMR)** en `http://localhost:3000`. Cualquier cambio en archivos `.tsx` o `.ts` se reflejará instantáneamente sin reiniciar el servidor.

---

2. Estructura del Código y Organización de Módulos

```
AegisAI/
├── config/
│   └── frameworks/            # Almacenamiento local JSON de marcos BYOF (LFPDPPP, LFPC, EU AI Act)
├── docs/                      # Suite documental oficial de ingeniería y manuales
├── public/                    # Archivos estáticos servidos en la raíz del servidor
├── scripts/                   # Suites automatizadas de pruebas (OWASP AI, Integración) y generador
├── src/
│   ├── components/
│   │   ├── admin/             # Panel de administración de modelos y tokens (AdminSettingsView)
│   │   ├── api/               # Documentación y prueba de endpoints API (ApiConnectionView)
│   │   ├── architecture/      # Diagramas y especificación de arquitectura (ArchitectureView)
│   │   ├── auth/              # Inicio de sesión seguro con Google OAuth (Login)
│   │   ├── dashboard/         # Vistas principales: Dashboard, AuditForm, CatalogView, AuditReportV
│   │   ├── gcp/               # Escáner en vivo de Google Cloud (GcpLiveScannerView)
│   │   ├── layout/            # Componentes estructurales: Sidebar, BrandAssets
│   │   └── ui/                # Sistema de diseño: Button, Badge, Card, Input, Label
│   ├── lib/
│   │   ├── actionPlanGenerator.ts # Generador de planes de acción en Markdown y PDF
│   │   ├── aisvsData.ts       # Definición canónica de los 12 capítulos de OWASP AI-SVS 1.0
│   │   ├── data.ts            # Datos iniciales y utilitarios
│   │   ├── firebase.ts        # Inicialización del cliente de Firebase y Firestore
│   │   ├── iso42001Data.ts    # Definición canónica de ISO/IEC 42001:2023
│   │   ├── remediatedArchitectureGenerator.ts # Generador de arquitectura remediada al 100%
│   │   ├── sampleAudits.ts    # Auditorías de ejemplo preconfiguradas
│   │   └── utils.ts           # Funciones auxiliares de formateo y estilo
│   ├── services/
│   │   └── gcpScannerService.ts # Servicio de escaneo en vivo de Google Cloud APIs
│   ├── types.ts               # Definiciones centrales de tipos e interfaces TypeScript
│   ├── App.tsx                # Enrutador principal de vistas con React.lazy y Suspense
│   └── main.tsx               # Punto de entrada de la aplicación React
├── firestore.rules            # Reglas declarativas de seguridad de Cloud Firestore
├── server.ts                  # Backend Express: Motor Map-Reduce, Inferencia Gemini y API REST
├── package.json               # Dependencias y scripts de construcción
├── tsconfig.json              # Configuración del compilador TypeScript (Modo Estricto)
└── vite.config.ts             # Configuración del empaquetador Vite y división de chunks
```

3. Convenciones de Codificación y Mejores Prácticas

3.1 Tipado Estricto en TypeScript

- No se permite el uso del tipo `any` en funciones del núcleo evaluador.
- Toda nueva propiedad añadida a `AuditReport` o `PuntoProbado` debe definirse formalmente en `src/types.ts`.
- La verificación de tipos `npm run lint` (`tsc --noEmit`) debe ejecutarse y pasar con 0 errores antes de enviar cualquier commit.

3.2 Manejo de Respuestas de Inteligencia Artificial

Al invocar modelos de Gemini para tareas estructuradas:

- Debe utilizarse **siempre** `responseSchema` con tipos primitivos (`Type.OBJECT`, `Type.ARRAY`, `Type.STRING`).
- Los parámetros de inferencia deben ser estrictamente deterministas:

```
config: {
  temperature: 0.0,
  topK: 1,
  topP: 0.0,
  responseMimeType: 'application/json',
  responseSchema: miEsquemaEstricto
}
```

3.3 Gestión de Streaming Server-Sent Events (SSE)

En endpoints que transmiten eventos SSE:

- Siempre implementar el pulso `: keep-alive\n\n` mediante `setInterval` para evitar que balanceadores de carga o proxies corten la conexión.
- Capturar el evento `req.on('close')` para limpiar temporizadores y evitar fugas de memoria.

---

4. Cómo Extender el Sistema

4.1 Incorporar una Nueva Plataforma de Nube a la Matriz de Responsabilidad Compartida

Para añadir un nuevo proveedor cloud (ej. *Oracle Cloud Infrastructure - OCI Generative AI* o *IBM watsonx*):

1. Abra `src/types.ts` y amplíe el tipo `TechPlatformKey`:

```
export type TechPlatformKey = 'gemini-enterprise' | 'vertex-ai' | 'azure-openai' | 'aws-bedroc
```

2. Abra `server.ts` y añada la entrada en el diccionario `PLATFORM_PROFILES`:

```
'oci-ai': {
  name: 'Oracle Cloud OCI Generative AI',
  deploymentType: 'PaaS',
  provider: 'Oracle Cloud Infrastructure',
  inheritedSafeguards: [
    'Aislamiento de instancias dedicadas de inferencia de modelos.',
    'Certificación SOC 2 Type II y cumplimiento ISO 27001.',
    'Garantía contractual de no retención de prompts de clientes para reentrenamiento.'
  ],
  exemptChapters: ['C04', 'C06']
}
```

3. Abra `src/components/dashboard/AuditForm.tsx` y añada la tarjeta en la Sección 3 del formulario.

4.2 Extender el Esquema de Recetas de Mitigación (4D)

Si se desea añadir un nuevo campo al plan de acción (ej. `impactLevel` o `cveReferences`):

1. Modifique la interfaz `RemediationDetails` en `src/types.ts`.
2. Actualice el esquema `batchResponseSchema` en `server.ts` para que Gemini genere el nuevo campo estructuradamente.

3. Actualice `src/components/dashboard/AuditReportView.tsx` y `src/lib/actionPlanGenerator.ts` para renderizar el nuevo atributo en la interfaz y en el PDF exportable.

4.3 Añadir o Modificar Proveedores en el Motor Multi-Proveedor LLM

Para soportar un nuevo protocolo o enrutador de inferencia:

1. Modifique el tipo `LLMProviderType` en `src/types.ts`:

```
export type LLMProviderType = 'gemini' | 'openrouter' | 'openai' | 'custom_openai_compatible'
```

2. En `server.ts`, actualice la función despachadora `executeAIWithFallback` para conectar el cliente correspondiente.
3. En `src/components/admin/AdminSettingsView.tsx`, añada la pestaña visual del nuevo proveedor, con sus modelos sugeridos y URL base por defecto.
4. Valide la integración ejecutando la suite de pruebas:

```
npm run test:llm
```

---

5. Pruebas y Validación Automatizada

AegisAI cuenta con 4 suites de validación automatizada en la carpeta `scripts/`:

```
# 1. Comprobación estricta de tipos de TypeScript
npm run lint

# 2. Batería de motores LLM multi-proveedor y conectividad
npm run test:llm

# 3. Ejecución de la suite de integración funcional y normativas
npm run test:integration

# 4. Pruebas de integración del Escáner en Vivo de Google Cloud (GCP Live Scanner)
npm run test:gcp

# 5. Ejecución de la suite de ciberseguridad OWASP AI Top 10
npm run test:security

# 6. Verificación de construcción limpia de producción (Frontend + Backend)
npm run build
```

---

6. Mantenimiento y Actualizaciones del Sistema

6.1 Actualización del SDK de Google Gemini (`@google/genai`)

AegisAI utiliza el SDK oficial unificado `@google/genai`. Para actualizar a versiones superiores:

```
npm install @google/genai@latest
npm run lint
npm run build
```

Tras la actualización, ejecute `/api/admin/test-connection` para validar la compatibilidad de las llamadas de inferencia.

6.2 Flujo de Ramas y Control de Versiones (Git Workflow)

- `main`: Rama de producción protegida. Solo recibe cambios validados mediante Pull Request con las 3 suites de pruebas aprobadas (100%).
- `develop`: Rama de integración para desarrollo continuo.
- `feature/*`: Ramas de características nuevas (ej. `feature/iso-5338-integration`).
- `hotfix/*`: Correcciones inmediatas de seguridad.